DL_T14_2D — Sentiment Analysis of Code-Mixed Telugu–English Mobile-Network Complaints

Task 2, Team 14, variant 2D · CM701PC / CM751PC Deep Learning, MGIT (Autonomous) · Code: github.com/kareem1207/2D-Mobile-network-customer-complaints

Roll no.	Role (department allotment)	Contribution (to be confirmed by the team before submission)
23261A6646	Data lead	Dataset generator, annotation guide, data card, error analysis
23261A6655	Model lead	Models, experiments, README / reproducible code
23261A6657	Evaluation & report lead	Metrics, plots, target checks, report

1. Problem understanding

Customers write Telugu in English letters mixed with English words ("network chala bagundi but bill lo extra charges vesaru"), often with polite openings and the real complaint last. Three classes are needed: negative, positive and mixed. Targets: macro-F1 ≥ 0.65, F1 on "mixed" ≥ 0.45, predicting 10,000 reviews in < 60 s on CPU, and a defended maximum sequence length. The trade-off is context against speed: longer sequences and bidirectional layers read more but cost time (prediction time grows about linearly with length, Table 1). With very little labelled data, a small model (64-unit layer, 64-d embeddings, 56k–114k parameters) is also the safer choice against overfitting.

Important caveat. No public labelled Telugu–English mobile-network corpus exists, so the dataset is **synthetic**, generated from hand-written phrase banks. All numbers below measure behaviour on this generator, not real customers; they are likely optimistic.

2. Data preparation and exploratory analysis

3,000 reviews (1,000 per class), split 2,100/450/450 train/validation/test with seed 42, no text shared between splits, vocabulary (746 types) built on train only. Length is 2–83 tokens (median 20); mixed reviews are longer (mean 38 vs 21 tokens), a shortcut we flag in §6. Reviews fall in six structures (plain, polite opening, long with buried complaint, sarcasm, negation such as "problem ledu", and distant mixed) and half carry injected spelling variation. To test generalisation, the last 3 of 10 phrases per aspect and polarity are *held out* of training, and half of the validation and test reviews use them. The annotation guide and data card are in the repository; labels are assigned by construction, not by humans.

3. Models and experiments

Embedding(64, masked) → recurrent layer(64) → Dropout(0.3) → Softmax(3); Adam, early stopping on validation loss, 3 seeds per configuration. **Sequence length (LSTM, validation F1, Table 1):**

Max length	16	32	64	128
Val F1, keep last tokens	0.847	0.850	0.854	0.850
Val F1, keep first tokens	0.638	0.706	0.846	0.850
Seconds / 10k reviews	0.29	0.49	0.90	1.61

Keeping the *last* tokens is nearly flat from 16 to 128 tokens, so we chose **64** (best mean, 0.854; 128 costs 1.8× the time for no gain). Keeping the *first* tokens at 16 loses 0.21 F1 because the complaint is at the end. **Layers and training strategies (test split, mean over 3 seeds, length 64):**

Layer	Start	Val F1	Test macro-F1 (mean ± sd)	Mixed F1	s / 10k	Params
SimpleRNN	scratch	0.866	0.830 ± 0.011	0.985	0.47	56,323
SimpleRNN	IMDB init	0.863	0.834 ± 0.014	0.980	0.46	56,323
LSTM	scratch	0.854	0.818 ± 0.018	0.998	0.92	81,091
LSTM	IMDB init	0.849	0.814 ± 0.008	0.968	0.98	81,091
GRU	scratch	0.832	0.775 ± 0.013	0.926	0.82	73,027
GRU	IMDB init	0.855	0.813 ± 0.027	0.956	0.82	73,027
BiLSTM	scratch	0.839	0.791 ± 0.022	0.979	1.14	114,307
BiGRU	scratch	0.866	0.832 ± 0.020	0.983	0.87	98,179

"IMDB init" starts from an English IMDB model of the same layer (IMDB test accuracy: SimpleRNN 0.615, LSTM 0.840, GRU 0.859): recurrent weights are copied, embedding rows are copied for the 21% of our vocabulary that also occurs in the IMDB vocabulary, and a new 3-class head is trained. SimpleRNN, LSTM and BiGRU (0.814–0.834) are within seed noise of each other; GRU from scratch is lower (0.775).

4. Results against every target

Target	Required	Best model (SimpleRNN, scratch, seed 1)	Met?
Macro-F1, test	≥ 0.65	0.842 (all configurations 0.775–0.834 mean)	Yes
F1 of "mixed" class	≥ 0.45	0.993 (suspiciously easy, see §6)	Yes
Predict 10,000 reviews (4-core CPU)	< 60 s	0.55 s (slowest model 1.14 s)	Yes
Maximum sequence length	defend	64 tokens, from the sweep above	Done

The best configuration was picked by mean validation F1 (SimpleRNN scratch 0.866; BiGRU 0.866 and SimpleRNN with IMDB init 0.863 are statistically tied), and its best-validation seed is reported. Per class F1: negative 0.760, positive 0.773, mixed 0.993; Cohen's κ of model vs. gold = 0.763.

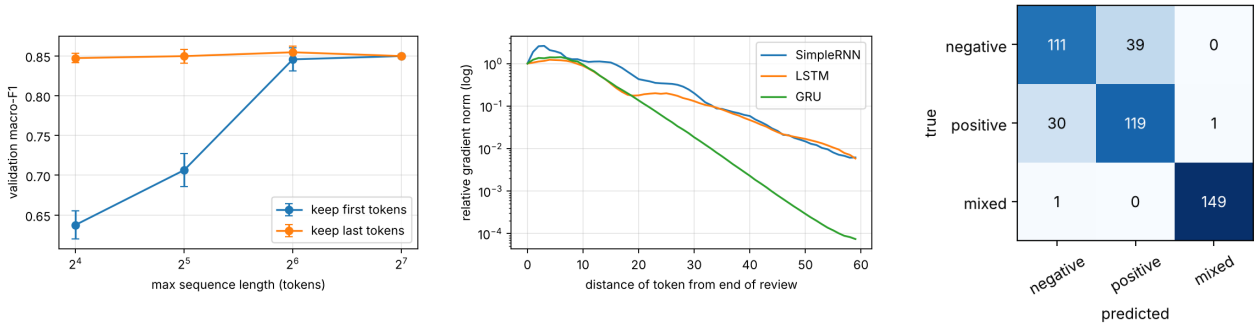


Figure: left, length sweep; centre, gradient reaching earlier tokens; right, test confusion matrix of the best model.

5. Answers to the report questions

Q1 (unfolded graph, vanishing gradients). Unfolding a recurrent net over T tokens multiplies T Jacobians, so a token d steps from the end influences the output through roughly d factors; LSTM/GRU gates add an (almost) additive path that should keep gradients alive. **Our data did not show the textbook result:** SimpleRNN was not worse on long reviews (F1 on reviews > 40 tokens: SimpleRNN 0.864, LSTM 0.853, GRU 0.770). We measured the gradient norm of the loss w.r.t. each token embedding on long reviews; relative to the last token it fell to 0.058 (SimpleRNN), 0.047 (LSTM) and 0.002 (GRU) at 40 tokens from the end. All three decay, and the gated GRU decays most. A likely reason is that in this synthetic data the decisive words sit in the last ~20 tokens and even "distant mixed" reviews are caught by connectors such as *kani* or *but*, so no layer has to remember far back. The accuracy figures therefore do **not** support the claim that gates help here.

Q2 (English IMDB start). Mixed evidence. Change in test macro-F1 vs. scratch: SimpleRNN +0.004, LSTM –0.004, GRU +0.038 (sd 0.013–0.027), mean +0.013. Only the GRU gain is large (about 3 sd of its scratch runs), but with 3 seeds and no similar gain for the other layers we treat it as unproven. The transfer is weak because only 21% of the vocabulary overlaps and the Romanised-Telugu words, which carry most sentiment here, have no English pre-trained meaning; the IMDB-trained SimpleRNN itself reached only 0.615 on IMDB. We conclude: no reliable benefit, small possible help for GRU.

Q3 (error analysis). The best model makes 71 errors on 450 reviews: 39 negative→positive, 30 positive→negative, and only 2 involve "mixed". **65 of 71 errors (92%) are on held-out phrases**: accuracy 0.974 on seen vs 0.700 on unseen wording, because new sentiment words (*impressive*, *okay*, *unstable*) are out-of-vocabulary. By structure: negation 0.66 (17 of 50 wrong), plain 0.83, polite opening 0.84, sarcasm 0.84 (3 of 19 wrong), long 0.87, distant mixed 1.00; clean and noisy spelling are similar (0.83 vs 0.85). Among sampled errors, items such as "network busy message ee madhya raaledu" (positive by absence of a complaint) are only weakly positive, so some error is label ambiguity, but we cannot quantify it: inter-annotator κ has **not** been measured because labels are programmatic. A blind 210-item sheet and `src/kappa.py` are provided; until a second human

fills it, the error share caused by unclear labels is unknown. The one design choice with the largest measured effect is keeping the *last* tokens (0.847 vs 0.638 validation F1 at length 16, $\Delta = 0.209$; Table 1).

6. Limitations and future work

(i) Synthetic, template-based data: real reviews have more vocabulary, slang, emojis, Telugu script and spelling noise; accuracy on real data will be lower. (ii) Shortcuts: longer reviews and contrastive connectors make “mixed” easy (F1 0.99) and probably inflate it. (iii) Word-level vocabulary cannot read unseen spellings; character n-grams or a multilingual pre-trained encoder would help. (iv) No human second annotator yet. With more time we would collect real complaints with consent, double-annotate them, and re-run the layer comparison on data where long-range context is genuinely needed.

7. References

[1] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, Ch. 7, 9, 10. [2] A. Maas et al., “Learning word vectors for sentiment analysis,” ACL, 2011 (IMDB). [3] Dravidian CodeMix shared task, FIRE workshop. [4] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, 1997. [5] K. Cho et al., “Learning phrase representations using RNN encoder-decoder,” EMNLP, 2014.

Appendix: use of AI tools

The dataset generator, experiment code, notebook and first draft of this report were produced with Claude Code (Anthropic) at the request of the data lead. Every number in this report is read from files in `results/` written by the submitted code (seeds fixed). Each member must read and understand the code before the viva; the contribution table above is a draft that the team must correct to reflect who actually did what.